



## Documentación de Oracle Cloud Infrastructure

[Probar cuenta gratuita](#)

[Servicios de infraestructura](#) > [API Gateway](#) > [Adición de autenticación y autorización a despliegues de API](#) >

≡ Todas las páginas

Actualizado 2026-06-25

# Validación de tokens para agregar autenticación y autorización a despliegues de API

Puede agregar la funcionalidad de autenticación y autorización a un gateway de API haciendo que el propio gateway de API valide los tokens incluidos en una solicitud (como se describe en este tema). También puede hacer que el gateway de API transfiera un token de acceso de varios argumentos o de un solo argumento incluido en una solicitud a una función de autorizador desplegada en OCI Functions para su validación (como se describe en [Transferencia de tokens a funciones de autorizador para agregar autenticación y autorización a despliegues de API](#)).

Para que el gateway de API valide el token incluido en una solicitud, cree una política de solicitud de autenticación de tipo TOKEN\_AUTHENTICATION. Los tokens que utiliza para controlar el acceso a las API son, a menudo, pero no siempre, tokens web JSON (JWT).

Al utilizar una política TOKEN\_AUTHENTICATION, permite que un despliegue de API utilice tokens para la autenticación y autorización mediante la inclusión de dos tipos diferentes de política de solicitud en la especificación de despliegue de API:

- Una política de solicitud de autenticación para todo el despliegue de API que especifica el uso de tokens, incluyendo cómo validarlos y si los usuarios finales no autenticados pueden acceder a rutas en el despliegue de API.

- Una política de solicitud de autorización para cada ruta que especifica las operaciones que puede realizar un usuario final, según los valores especificados para la reclamación `scope` de un JWT.

Puede agregar políticas de solicitud de autorización y autenticación a una especificación de despliegue de API mediante las siguientes opciones:

- [Uso de la consola](#)
- [Edición de un archivo JSON.](#)

### **Nota**

En versiones anteriores, creaba políticas de solicitud de autenticación de tipo `JWT_AUTHENTICATION` para utilizar los JWT para la autenticación. Las políticas de solicitud de autenticación de `JWT_AUTHENTICATION` existentes aún se admiten (consulte [Notas sobre el uso de políticas de solicitud de JWT AUTHENTICATION](#)). Sin embargo, si está creando nuevas políticas de solicitud de autenticación para autenticar JWT, le recomendamos que cree políticas de solicitud de autenticación como políticas `TOKEN_AUTHENTICATION`. El uso de políticas `TOKEN_AUTHENTICATION` le permite:

- Valide los tokens JWT y los tokens que no sean JWT.
- Valide los tokens mediante un proveedor de identidad para obtener un punto final de introspección.
- Especifique políticas de fallo de validación, incluida la generación de un nuevo token de JWT en caso de que falte un token de JWT o no sea válido en la solicitud original.

## ¿Qué sucede durante la autenticación de token?

Cuando un gateway de API recibe una solicitud de un cliente de API y usted ha especificado una política de autenticación de token, el gateway de API localiza un token (por ejemplo, en una cabecera de token) y utiliza ese token.

Especifique cómo el gateway de API valida el token que ha obtenido definiendo la política de validación de la política de autenticación de token como uno de los siguientes tipos:

- **OAuth Punto final de introspección 2.0:** especifique este tipo de política de validación si desea que el gateway de API valide un token JWT o no JWT con el punto final de introspección de un proveedor de identidad. Debe especificar la URL de detección del proveedor de identidad desde el que obtener el punto final de introspección. En este caso, el gateway de API transfiere

las credenciales de cliente (el ID de cliente, junto con el secreto de cliente recuperado del servicio Vault) al proveedor de identidad para validar el token. El token se valida sin el uso de claves públicas. Para acelerar la validación futura, puede especificar que desea que el gateway de API almacene en caché la respuesta del punto final de introspección durante entre 1 hora (valor por defecto) y 24 horas. Si está definiendo la especificación de despliegue de API en un archivo JSON y desea este comportamiento, incluya una política de validación de tipo

`REMOTE_DISCOVERY` .

- **JWKS remoto:** especifique este tipo de política de validación si desea que el gateway de API utilice claves de verificación públicas recuperadas de un proveedor de identidad en tiempo de ejecución para verificar un JWT. En este caso, el gateway de API se pone en contacto con el proveedor de identidad para verificar el JWT. El proveedor de identidad actúa como servidor de autorización. Si está definiendo la especificación de despliegue de API en un archivo JSON y desea este comportamiento, incluya una política de validación de tipo `REMOTE_JWKS` .
- **Claves estáticas:** especifique este tipo de política de validación si desea que el gateway de API utilice claves de verificación públicas ya emitidas por un proveedor de identidad (denominadas "claves estáticas") para verificar un JWT. En este caso, el gateway de API puede verificar el JWT localmente en tiempo de ejecución sin tener que ponerse en contacto con el proveedor de identidad. El resultado es una validación de token más rápida. Si está definiendo la especificación de despliegue de API en un archivo JSON y desea este comportamiento, incluya una política de validación de tipo `STATIC_KEYS`

Si la validación se realiza correctamente, el gateway de API enruta la solicitud al punto final de API adecuado.

Si la validación falla debido a un token no válido o faltante en la solicitud original, especifique el comportamiento del gateway de API definiendo la política de fallos de validación de la política de autenticación de token como uno de los siguientes tipos:

- **Por defecto (HTTP 401 no autorizado):** especifique este tipo de política de fallo de validación si desea que el gateway de API devuelva una respuesta HTTP-401 al cliente de API. Esta es la respuesta por defecto. Si está definiendo la especificación de despliegue de API en un archivo JSON y desea este comportamiento, simplemente no incluya una política de fallo de validación.
- **Respuesta personalizada:** especifique este tipo de política de fallo de validación si desea que el gateway de API devuelva una respuesta alternativa (una respuesta modificada) al cliente de API, en lugar de una respuesta HTTP-401. Si está definiendo la especificación de despliegue de API en un archivo JSON y desea este comportamiento, incluya una política de fallo de validación de tipo `MODIFY_RESPONSE` .
- **OAuth 2.0:** especifique este tipo de política de fallo de validación si desea que el gateway de API obtenga un token de JWT nuevo y válido del proveedor de identidad para las solicitudes GET (habiendo almacenado primero de forma segura los parámetros de consulta de solicitud

originales). Para las solicitudes PUT y POST, puede especificar una ruta de acceso relativa en el despliegue de API actual al que redirigir clientes de API.

Con este tipo de política de fallos de validación, también puede configurar los siguientes backends:

- **Backend de conexión:** si lo desea, especifique una ruta de redireccionamiento de conexión estática para identificar una ruta en la especificación de despliegue de API que utilice un backend de conexión. El proveedor de identidad redirige a los clientes de API a esta ruta como parte del flujo de autenticación y, a continuación, el gateway de API redirige a los clientes de API a la ruta que solicitaron originalmente. Consulte [Adición de conexión como backend de gateway de API](#).
- **Backend de desconexión:** si lo desea, especifique una ruta de desconexión para identificar una ruta en la especificación de despliegue de API que utilice un backend de desconexión. El gateway de API elimina las cookies de sesión asociadas, revoca los tokens llamando al punto final de la sesión final del proveedor de identidad y redirige a una URL posterior a la desconexión permitida. Consulte [Adición de desconexión como backend de gateway de API](#).

Si está definiendo la especificación de despliegue de API en un archivo JSON y desea un comportamiento de fallo de validación de OAuth 2.0, incluya una política de fallo de validación de tipo `OAuth2`.

Tenga en cuenta que las políticas de fallo de validación de tipo OAuth 2.0 actualmente solo soportan el flujo de autorización de OpenID Connect y solo la generación de token de JWT (consulte [Notas sobre OAuth 2.0 y OpenID Connect](#)). En el caso del flujo de autorización de OpenID Connect, se devuelven dos tokens denominados `id_token` (siempre codificados en JWT) y `access_token` (pueden estar codificados en JWT). El gateway de API guarda los valores de token en las variables de contexto `request.auth[id_token]` y `request.auth[access_token]`, junto con las reclamaciones personalizadas en las variables de contexto `request.auth[id_token_claims][<claim-name>]` y `request.auth[access_token_claims][<claim-name>]`, respectivamente (consulte [Adición de variables de contexto a políticas y definiciones de backend HTTP](#)). Al recibir el nuevo token de JWT `id_token`, el gateway de API guarda los valores de token en el contexto de solicitud. Si no ha configurado una ruta de redireccionamiento de conexión estática, el gateway de API seguirá procesando la solicitud original del cliente de API mediante el token. Si ha configurado una ruta de redireccionamiento de conexión estática, el cliente de API vuelve del proveedor de identidad a la ruta que utiliza el backend de conexión. A continuación, el gateway de API utiliza los detalles de solicitud intermedia guardados para redirigir al cliente de API de nuevo a la ruta que el cliente de API solicitó originalmente.

La ubicación desde la que el gateway de API obtiene un token depende tanto del tipo de política de validación (una de las **OAuth 2.0 introspection endpoint**, **Remote JWKS** o **Static keys**) como del tipo de política de fallo de validación (una de las siguientes opciones: **Default (HTTP 401**

## Unauthorized), Custom response o OAuth 2.0:

- Si especifica una política de validación de tipo **OAuth 2.0 punto final de introspección** y una política de fallo de validación de tipo **OAuth 2.0**, el gateway de API inicialmente intenta obtener el token de uno u otro de los siguientes elementos:
  - Si ha seleccionado la opción **Usar Cookies para Sesión** en la política de fallos de validación, desde una cookie de sesión.
  - De lo contrario, desde el encabezado X-APIGW-TOKEN en la solicitud.

Si el gateway de API no puede obtener un token de la ubicación inicial, el gateway de API obtiene el token de la cabecera de solicitud o el parámetro de consulta que especifique en la política de autenticación de token.

Si la validación de token se realiza posteriormente correctamente y ha seleccionado la opción **Usar cookies para sesión**, el gateway de API almacena el token generado como una cadena no legible por el usuario en una cookie de sesión. Si el cliente de API realiza una solicitud posterior, el token se obtiene de la cookie de sesión. Para evitar ataques CSRF, cuando el TOKEN generado se almacena en una cookie de sesión, el gateway de API también devuelve un TOKEN CSRF en una cabecera de respuesta X-CSRF-TOKEN (consulte [Notas sobre la protección contra la falsificación de solicitudes entre sitios \(CSRF\)](#)).

- Si no especifica una política de validación de tipo **OAuth 2.0 punto final de introspección** y una política de fallo de validación de tipo **OAuth 2.0**, el gateway de API obtiene el token de la cabecera de solicitud o del parámetro de consulta que especifique en la política de autenticación de token.

## Notas sobre los tokens web JSON (JWT)

Los tokens que utiliza para controlar el acceso a las API suelen ser tokens web JSON (JWT). Un JWT es un token de acceso basado en JSON enviado en una solicitud HTTP de un cliente de API a un recurso. Los proveedores de identidad emiten los JWT. API Gateway soporta el uso de cualquier proveedor de identidad compatible con OAuth2, como OCI IAM con dominios de identidad, Oracle Identity Cloud Service (IDCS), Auth0 y Okta. Si se necesita un JWT para acceder a un recurso, el recurso valida el JWT con un servidor de autorización mediante una clave de verificación pública correspondiente, ya sea llamando a un punto final de validación en el servidor de autorización o mediante una clave de verificación local proporcionada por el servidor de autorización.

Un JWT comprende:

- Una cabecera que identifica el tipo de token y el algoritmo criptográfico utilizado para generar la firma.

- Una carga útil, que contiene reclamaciones sobre la identidad del usuario final y las propiedades del propio JWT. Una reclamación es un par clave-valor donde la clave es el nombre de la reclamación. Se recomienda una carga útil (aunque no es necesaria) para contener determinadas reclamaciones reservadas con nombres particulares, como tiempo de caducidad ( `exp` ), público ( `aud` ), emisor ( `iss` ) y "no antes de" ( `nbf` ). Una carga útil también puede contener reclamaciones personalizadas con nombres definidos por el usuario.
- Una firma para validar la autenticidad del JWT (derivada de base64 que codifica la cabecera y la carga útil).

Al utilizar JWT para controlar el acceso a las API, puede especificar que las reclamaciones reservadas en la carga útil del JWT deben tener valores concretos antes de que el gateway de API considere que el JWT es válido. Por defecto, los gateways de API validan los JWT utilizando las reclamaciones de caducidad ( `exp` ), público ( `aud` ) y emisor ( `iss` ), junto con la reclamación de "no antes de" ( `nbf` ) si está presente. También puede especificar valores aceptables para reclamaciones personalizadas. Consulte [Detalles del proveedor de identidad para usar en reclamaciones iss y aud y para el URI de JWKS](#).

Cuando se ha validado un JWT, el gateway de API extrae reclamaciones de la carga útil de los JWT como pares clave-valor y los guarda como registros en la tabla `request.auth` para que el despliegue de API las use. Por ejemplo, como variables de marco para su uso en una definición del backend HTTP (consulte [Agregación de variables de marco a políticas y definiciones de backend HTTP](#)). Si la carga útil del JWT contiene la reclamación `scope` , puede utilizar los valores de la reclamación en las políticas de solicitud de autorización para rutas individuales para especificar las operaciones que puede realizar un usuario final.

## Notas sobre OAuth 2.0 y OpenID Connect

El protocolo OAuth 2.0 permite la recuperación segura de recursos seguros al tiempo que protege las credenciales del cliente. OAuth 2.0 es un protocolo flexible y seguro que se basa en SSL (Secure Sockets Layer) para garantizar que los datos entre los servidores web y los navegadores permanezcan privados. Aunque OAuth 2.0 es diferente de JWT y se utiliza para diferentes fines, OAuth 2.0 y JWT son compatibles. Dado que el protocolo OAuth2 no especifica el formato de los tokens, los JWT se pueden incorporar al uso de OAuth2. Para obtener más información sobre OAuth 2.0, consulte la [documentación de OAuth 2.0 ↗](#).

OpenID Connect es una capa de identidad simple sobre el protocolo OAuth 2.0. El uso de OpenID Connect permite a los gateways de API verificar la identidad de un cliente de API según la autenticación realizada por un servidor de autorización. OpenID Connect también permite a los gateways de API obtener información de perfil básica sobre el cliente de API. Para obtener más información sobre OpenID Connect, consulte la [documentación de OpenID Connect ↗](#).

# Notas sobre la protección contra la falsificación de solicitudes entre centros (CSRF)

Un atacante puede montar un ataque CSRF aprovechando la existencia de una cookie del explorador para hacer que un usuario envíe un comando no deseado a una aplicación web, como un gateway de API. Si la aplicación determina que el usuario ya se ha autenticado correctamente debido a la existencia de la cookie del explorador, la aplicación ejecuta el comando con consecuencias potencialmente dañinas.

Al definir una política de validación de tipo **OAuth 2.0 punto final de introspección** y una política de fallo de validación de tipo **OAuth 2.0**, especifique cómo un gateway de API almacena un nuevo token de JWT que ha obtenido mediante el flujo de autorización de Connect OpenID:

- Seleccione la opción **Usar Cookies para la Sesión** si desea almacenar el nuevo token de JWT en una cookie de sesión. Para evitar posibles ataques CSRF, cuando el gateway de API almacena el TOKEN en una cookie de sesión, también devuelve un TOKEN CSRF en una cabecera de respuesta X-CSRF-TOKEN. Las solicitudes de mutación posteriores al gateway de API (como las solicitudes POST, PUT y DELETE, pero no las solicitudes GET) deben incluir el TOKEN CSRF en una cabecera de solicitud X-CSRF-TOKEN, además del TOKEN JWT en la cookie de sesión que se incluye automáticamente.

Tenga en cuenta que el gateway de API también almacena el token CSRF en la variable de contexto `request.auth[apigw_csrf_token]`. Si el cliente de API no puede leer la cabecera de respuesta inicial de X-CSRF-TOKEN por algún motivo, puede incluir la variable de contexto `request.auth[apigw_csrf_token]` en una política de respuesta de transformación de cabecera para agregar una cabecera de respuesta que contenga el token CSRF a cada respuesta devuelta al cliente de API. Este enfoque garantiza que el cliente de API pueda extraer correctamente el token CSRF de la cabecera de respuesta para su inclusión en las siguientes cabeceras de solicitud de mutación X-CSRF-TOKEN que envía al gateway de API. A continuación, se muestra un ejemplo de una política de respuesta de transformación de cabecera adecuada:

```
"responsePolicies": {
  "headerTransformations": {
    "setHeaders": {
      "items": [
        {
          "name": "MY-CSRF-TOKEN",
          "values": ["${request.auth[apigw_csrf_token]}"],
          "ifExists": "OVERWRITE"
        }
      ]
    }
  }
}
```

Para obtener más información sobre las políticas de respuesta de transformación de cabecera, consulte [Adición de Políticas de Respuesta de Transformación de Cabecera](#).

- No seleccione la opción **Usar Cookies para Sesión** si no desea almacenar el nuevo token de JWT en una cookie de sesión. En su lugar, el gateway de API devuelve un TOKEN no legible por humanos en una cabecera de respuesta X-APIGW-TOKEN. Las solicitudes posteriores al gateway de API deben incluir el mismo TOKEN en una cabecera de solicitud X-APIGW-TOKEN.

Para obtener más información sobre CSRF, busque en línea.

## Notas sobre el uso de políticas de solicitud JWT\_AUTHENTICATION

En versiones anteriores, es posible que haya creado políticas de solicitud de autenticación de tipo JWT\_AUTHENTICATION para utilizar JWT para la autenticación.

Si está creando nuevas políticas de solicitud de autenticación para utilizar JWT, ahora le recomendamos que cree políticas de solicitud de autenticación del tipo TOKEN\_AUTHENTICATION en su lugar (consulte [Uso de la consola para agregar políticas de solicitud de autorización y autenticación de token](#) y [Edición de un archivo JSON para agregar políticas de solicitud de autorización y autenticación de token](#)). También recomendamos migrar las políticas de solicitud de JWT\_AUTHENTICATION existentes a las políticas de TOKEN\_AUTHENTICATION.

Tenga en cuenta que las políticas de solicitud de JWT\_AUTHENTICATION existentes aún se admiten. Tenga en cuenta también que, aunque puede crear nuevas políticas de solicitud JWT\_AUTHENTICATION (como se describe en las instrucciones originales de la sección [Uso de una política de solicitud de autenticación JWT\\_AUTHENTICATION \(ya no se recomienda\)](#)), le recomendamos que cree políticas de solicitud de autenticación de tipo TOKEN\_AUTHENTICATION en su lugar.

---

¿Le ha resultado útil este artículo?

